LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 1

Android Basics With Compose

Oleh:

Muhammad Guntur Ricky Adhitya NIM. 2410817310003

PROGRAM STUDI TEKNOLOGI INFORMASI

FAKULTAS TEKNIK

UNIVERSITAS LAMBUNG MANGKURAT

APRIL 2026

LEMBAR PENGESAHAN

LAPORAN PRAKTIKUM PEMROGRAMAN MOBILE

MODUL 1

Laporan Praktikum Pemrograman Mobile Modul 1: Android Basics With Compose ini disusun
sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Muhammad Guntur Ricky Adhitya

NIM : 2410817310003

Menyetujui,
Asisten Praktikum

Muhammad Erza Raihan
NIM. 2310817210027

Menyetujui,
Dosen Penanggung Jawab Praktikum

Irham Maulani Abdul Gani, S.Kom., M.Kom.
NIP. 199710312025061009

DAFTAR ISI

LEMBAR PENGESAHAN.....2
DAFTAR ISI.....3
DAFTAR GAMBAR.....4
DAFTAR TABEL.....5
SOAL 1.....6
A. Source Code.....9
B. Output Program.....16
C. Pembahasan.....19
D. Essay.....19
DAFTAR PUSTAKA.....20

DAFTAR GAMBAR

Gambar 1.Tampilan Awal Aplikasi Modul 1.....6
Gambar 2. Tampilan Dadu Setelah Di-Roll Modul 1.....7
Gambar 3. Tampilan Roll Dadu Double Modul 1.....8
Gambar 4. Screenshot Output Soal 1 Compose Modul 1 (1).....16
Gambar 5. Screenshot Output Soal 1 Compose Modul 1 (2).....16
Gambar 6. Screenshot Output Soal 1 Compose Modul 1 (3).....17
Gambar 7. Screenshot Output Soal 1 Compose Modul 1 (4).....17
Gambar 8. Screenshot Output Soal 1 Compose Modul 1 (5).....18
Gambar 9. Screenshot Output Soal 1 XML Modul 1 (1).....18
Gambar 10. Screenshot Output Soal 1 XML Modul 1 (2).....19

DAFTAR TABEL

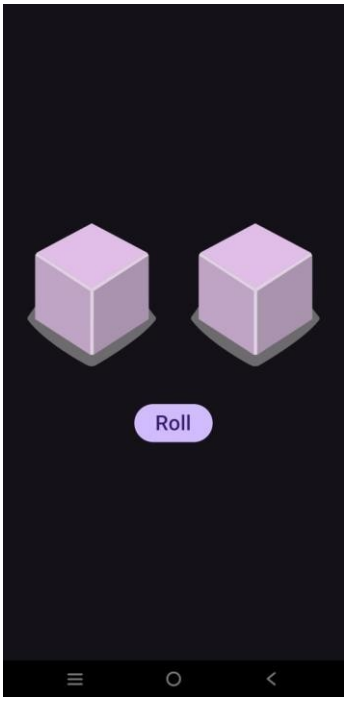
Tabel 1. Source Code MainActivity XML Soal 1 Modul 1 9
Tabel 2. Source Code layout XML MainActivity Modul 1 11
Tabel 3. Source Code MainActivity Compose Modul 1 12

SOAL 1

Soal Praktikum:

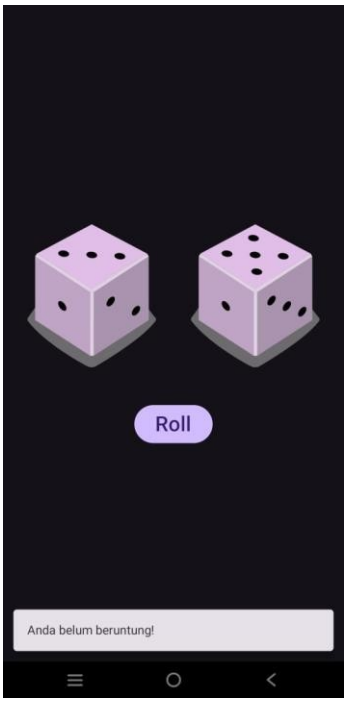
Buatlah sebuah aplikasi yang dapat menampilkan 2 buah dadu yang dapat berubah-ubah tampilannya pada saat user menekan tombol “Roll”. Aturan aplikasi yang akan dibangun adalah sebagaimana berikut:

1. Tampilan awal aplikasi setelah dijalankan akan menampilkan 2 buah dadu kosong seperti dapat dilihat pada Gambar 1.



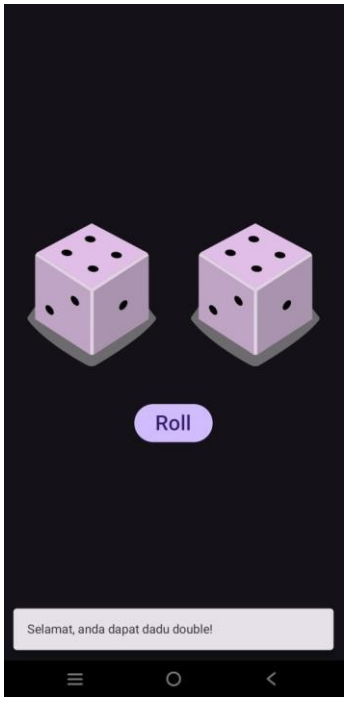
Gambar 1.Tampilan Awal
Aplikasi Modul 1

2. Setelah user menekan tombol “Roll” maka masing-masing dadu akan memperlihatkan sisi dadunya dengan angka antara 1 s/d 6. Apabila user mendapatkan nilai dadu yang berbeda antara Dadu 1 dengan Dadu 2, maka aplikasi akan menampilkan pesan “Anda belum beruntung!” seperti yang dapat dilihat pada Gambar 2.



Gambar 2. Tampilan Dadu
Setelah Di-Roll Modul 1

3. Apabila user mendapatkan nilai dadu yang sama antara Dadu 1 dan Dadu 2 atau nilai double, maka aplikasi akan menampilkan pesan “Selamat, anda dapat dadu double!” seperti yang dapat dilihat pada Gambar 3.



Gambar 3. Tampilan Roll Dadu Double Modul 1

4. Buatlah aplikasi tersebut menggunakan XML dan Jetpack Compose.
5. Upload aplikasi yang telah anda buat ke dalam repository GitHub ke dalam **folder Modul 1 dalam bentuk Project.** Jangan lupa untuk melakukan **Clean Project** sebelum mengupload pekerjaan anda pada repository.
6. Untuk gambar dadu dapat didownload pada link berikut: https://drive.google.com/file/d/14V3qXGdFnuoYN4AGd_9SgFh8kw8X9ySm/view?usp= sharing

Soal Essay:

1. Riset dan *cite* **artikel/jurnal/buku** mengenai Imperative Programming dan Declarative Programming beserta penerapannya di pembuatan UI. Buatkan opini paradigma apa yang kalian prefer berdasarkan riset yang sudah dilakukan. **
2. Apa yang membedakan OOP pada kotlin dengan OOP pada java? *
3. Sebutkan, dan jelaskan mengenai SOLID Principles *
4. Apa perbedaan antara Class dan Data Class pada Kotlin ?

**** = 2 sumber**
*** = 1 sumber**
Gunakanlah sumber primary artikel/jurnal/buku, web hanya sebagai penunjang opini/argumen

A. Source Code

Tabel 1. Source Code MainActivity XML Soal 1 Modul 1

```
package com.praktikum.dicerollerxml

import android.os.Bundle
import android.widget.ImageView
import android.widget.LinearLayout
import android.widget.Toast
import androidx.activity.enableEdgeToEdge
import androidx.appcompat.app.AppCompatActivity
import androidx.core.view.ViewCompat
import androidx.core.view.WindowInsetsCompat
import com.praktikum.dicerollerxml.databinding.ActivityMainBinding

class MainActivity : AppCompatActivity() {
    private lateinit var binding: ActivityMainBinding
    private var diceResults = listOf(0, 0)

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        binding = ActivityMainBinding.inflate(layoutInflater)
        setContentView(binding.root)
        updateUi(diceResults)
        binding.rollButton.setOnClickListener {
            diceResults = rollDice(diceResults.size)
            updateUi(diceResults)
            Toast.makeText(this, getString(checkLuck(diceResults)), Toast.LENGTH_SHORT).show()
        }
```

```
    }

    private fun updateUi(results : List<Int>) {
        binding.diceContainer.removeAllViews()
        results.forEach { result ->
            val diceView = ImageView(this).apply {
                layoutParams = LinearLayout.LayoutParams(
                    0, LinearLayout.LayoutParams.WRAP_CONTENT,
                    1.0f
                ).apply {
                    setMargins(8,8,8,8)
                }
                setImageResource(getDrawableDice(result))
                contentDescription = result.toString()
            }
            binding.diceContainer.addView(diceView)
        }
    }

    fun getDrawableDice(value: Int): Int = when (value) {
        0 -> R.drawable.dice_0
        1 -> R.drawable.dice_1
        2 -> R.drawable.dice_2
        3 -> R.drawable.dice_3
        4 -> R.drawable.dice_4
        5 -> R.drawable.dice_5
        else -> R.drawable.dice_6
    }
```

```
56
57      fun rollDice(count: Int = 2): List<Int> =
    List(count) {(1..6).random()}
58
59      fun checkLuck(results: List<Int>): Int =
60          if (results.all { it == results.first() })
    R.string.lucky
61          else R.string.badluck
62
63  }
```

Tabel 2. Source Code layout XML MainActivity Modul 1

```
1   <?xml version="1.0" encoding="utf-8"?>
2   <LinearLayout
3
    xmlns:android="http://schemas.android.com/apk/res/androi
    d"
4       android:layout_width="match_parent"
5       android:layout_height="match_parent"
6       android:gravity="center"
7       android:orientation="vertical"
8       android:padding="16.dp">
9
10      <LinearLayout
11          android:id="@+id/dice_container"
12          android:layout_width="match_parent"
13          android:layout_height="wrap_content"
14          android:orientation="horizontal">
15
16      </LinearLayout>
17
18      <Space
```

```
        android:layout_width="match_parent"
        android:layout_height="16dp" />

    <Button
        android:id="@+id/roll_button"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/roll" />

</LinearLayout>
```

Tabel 3. Source Code MainActivity Compose Modul 1

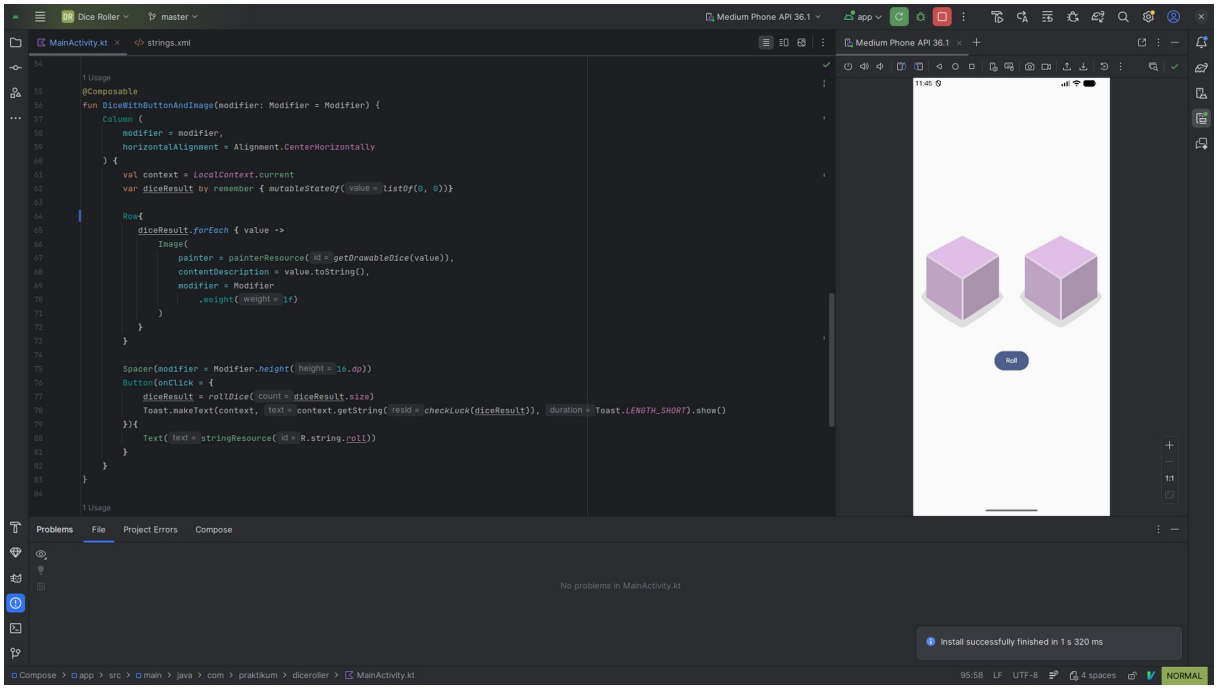
```
package com.praktikum.diceroller

import android.os.Bundle
import android.widget.Toast
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge
import androidx.compose.foundation.Image
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.height
import
androidx.compose.foundation.layout.wrapContentSize
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
```

```
19 import androidx.compose.ui.res.stringResource
20 import androidx.compose.ui.tooling.preview.Preview
21 import com.praktikum.diceroller.ui.theme.DiceRollerTheme
22 import androidx.compose.material3.Button
23 import androidx.compose.runtime.getValue
24 import androidx.compose.ui.res.painterResource
25 import androidx.compose.ui.unit.dp
26 import androidx.compose.runtime.mutableStateOf
27 import androidx.compose.runtime.remember
28 import androidx.compose.runtime.setValue
29 import androidx.compose.ui.platform.LocalContext
30
31 class MainActivity : ComponentActivity() {
32     override fun onCreate(savedInstanceState: Bundle?) {
33         super.onCreate(savedInstanceState)
34         enableEdgeToEdge()
35         setContent {
36             DiceRollerTheme {
37                 DiceRollerApp()
38             }
39         }
40     }
41 }
42
43
44
45 @Preview
46 @Composable
47 fun DiceRollerApp() {
48     DiceWithButtonAndImage(
49         modifier = Modifier
50             .fillMaxSize()
```

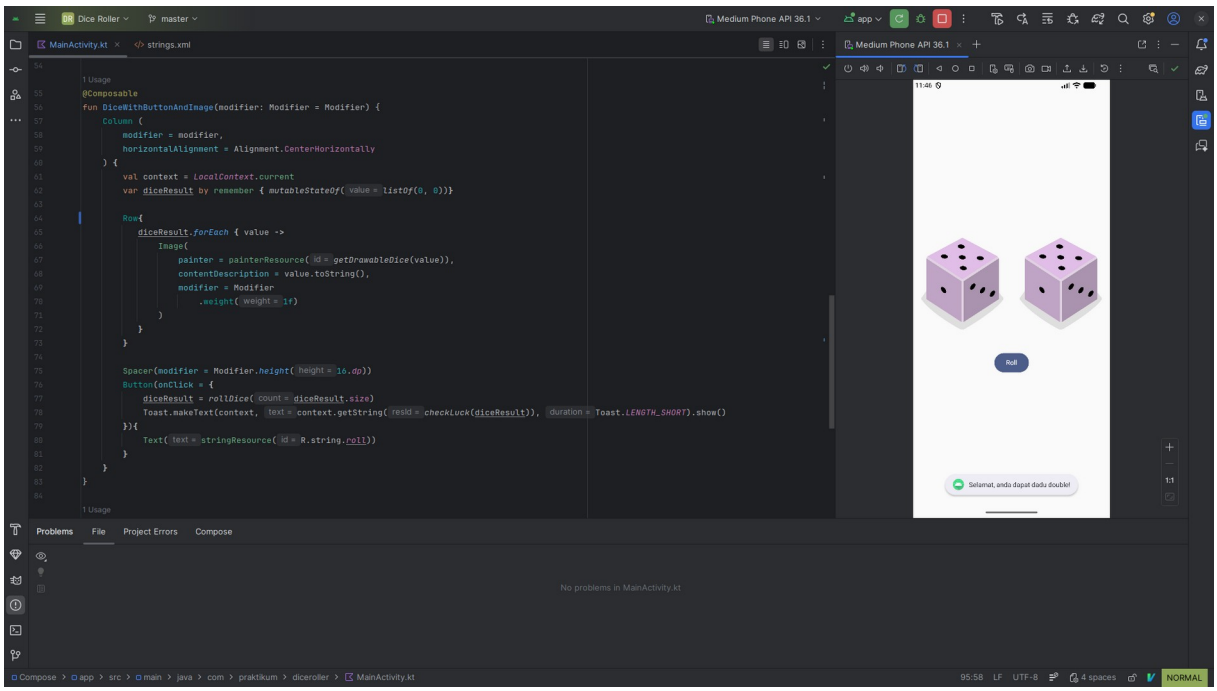
```
            .wrapContentSize(Alignment.Center)
    )
}

@Composable
fun DiceWithButtonAndImage(modifier: Modifier = Modifier) {
    Column (
        modifier = modifier,
                                horizontalAlignment = Alignment.CenterHorizontally
    ) {
        val context = LocalContext.current
                        var diceResult by remember { mutableStateOf(listOf(0, 0))}

        Row{
           diceResult.forEach { value ->
               Image(
                                                painter = painterResource(getDrawableDice(value)),
                                    contentDescription = value.toString(),
                   modifier = Modifier
                       .weight(1f)
               )
           }
        }

        Spacer(modifier = Modifier.height(16.dp))
        Button(onClick = {
            diceResult = rollDice(diceResult.size)
                                    Toast.makeText(context, context.getString(checkLuck(diceResult)),
```

```
Toast.LENGTH_SHORT).show()
        }){
            Text(stringResource(R.string.roll))
        }
    }
}

fun getDrawableDice(value: Int): Int = when (value) {
    0 -> R.drawable.dice_0
    1 -> R.drawable.dice_1
    2 -> R.drawable.dice_2
    3 -> R.drawable.dice_3
    4 -> R.drawable.dice_4
    5 -> R.drawable.dice_5
    else -> R.drawable.dice_6
}

fun rollDice(count: Int = 2): List<Int> = List(count) {(1 .. 6).random()}

fun checkLuck(result: List<Int>): Int =
    if(result.all {it == result.first()}) R.string.lucky
    else R.string.badluck
```

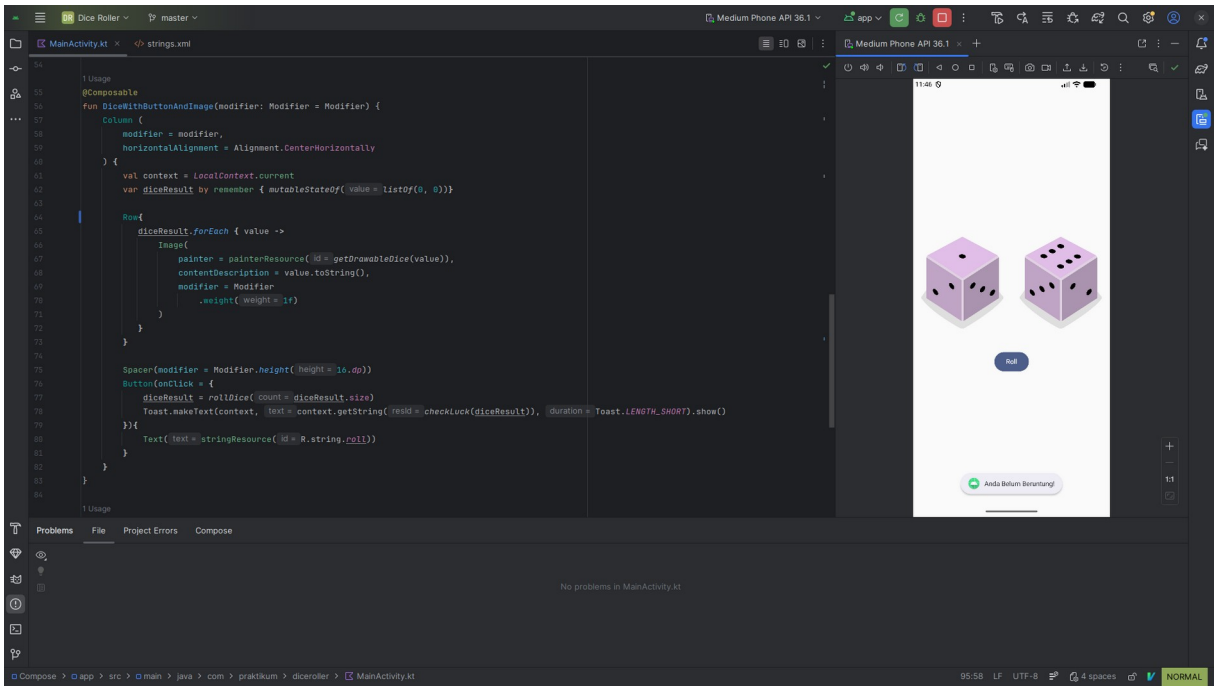
B. Output Program



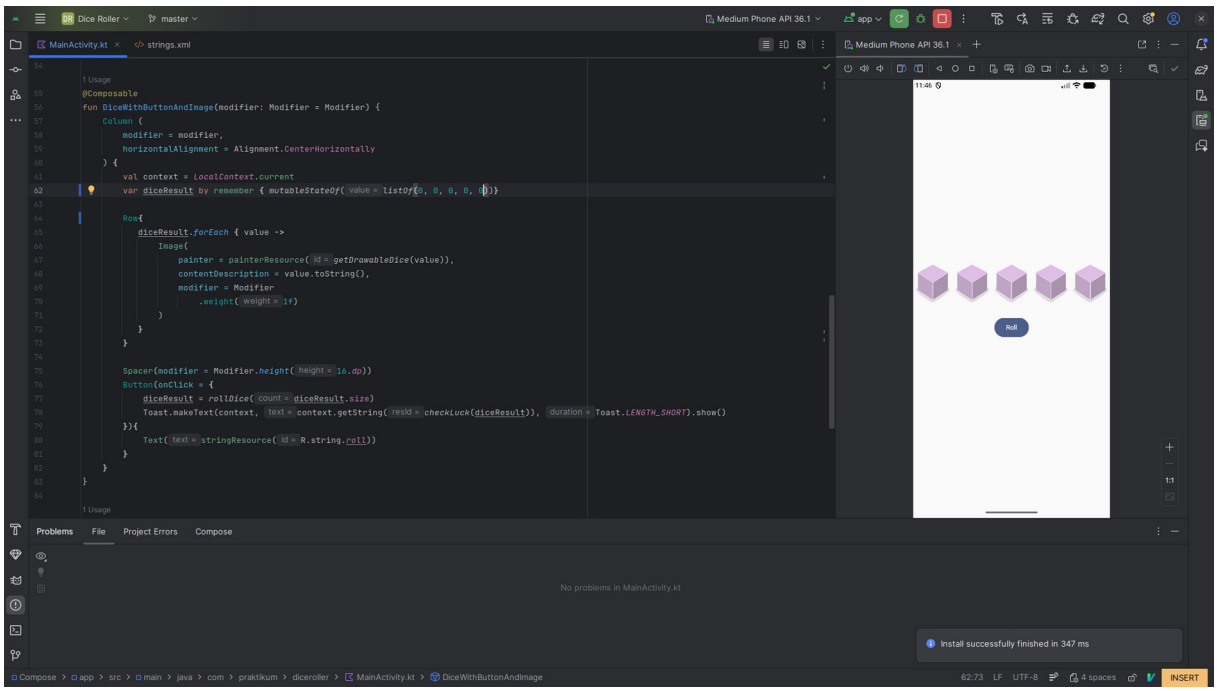
Gambar 4. Screenshot Output Soal 1 Compose Modul 1 (1)



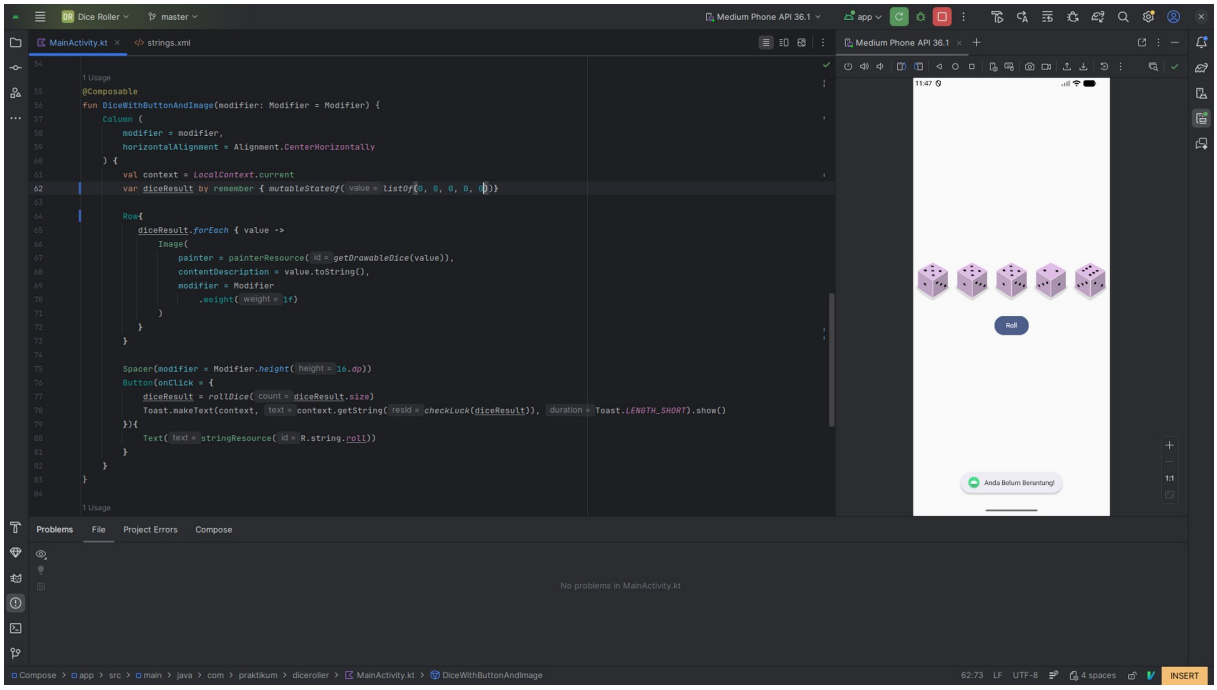
Gambar 5. Screenshot Output Soal 1 Compose Modul 1 (2)



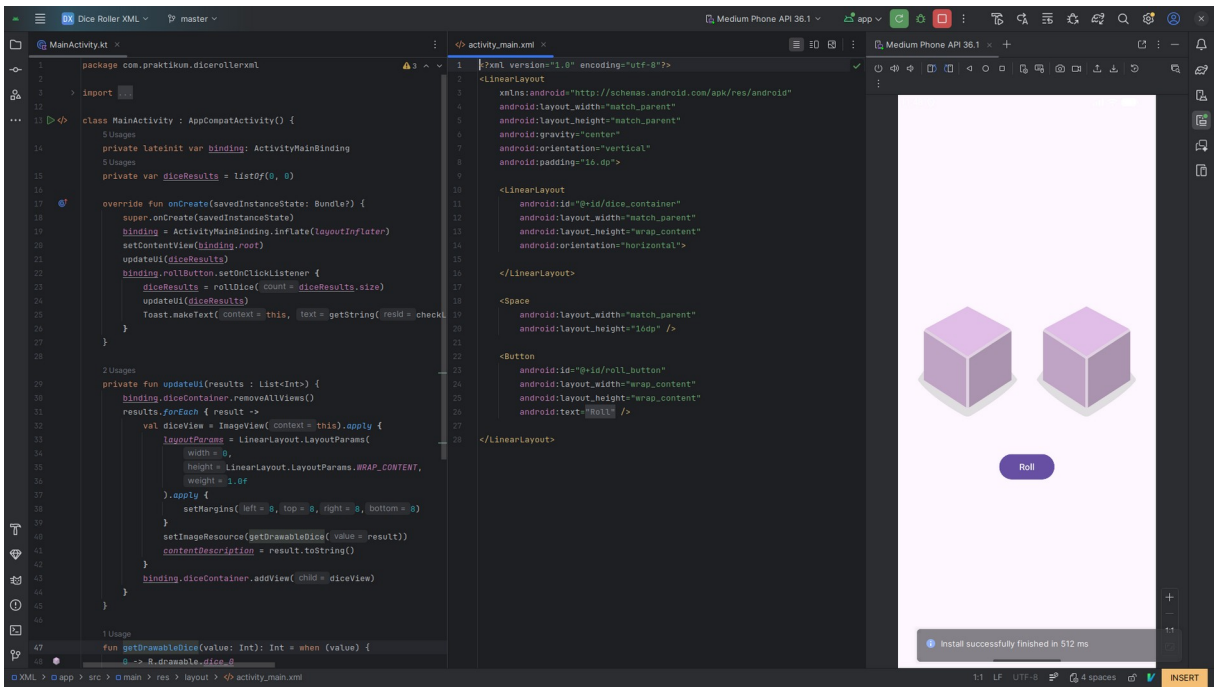
Gambar 6. Screenshot Output Soal 1 Compose Modul 1 (3)



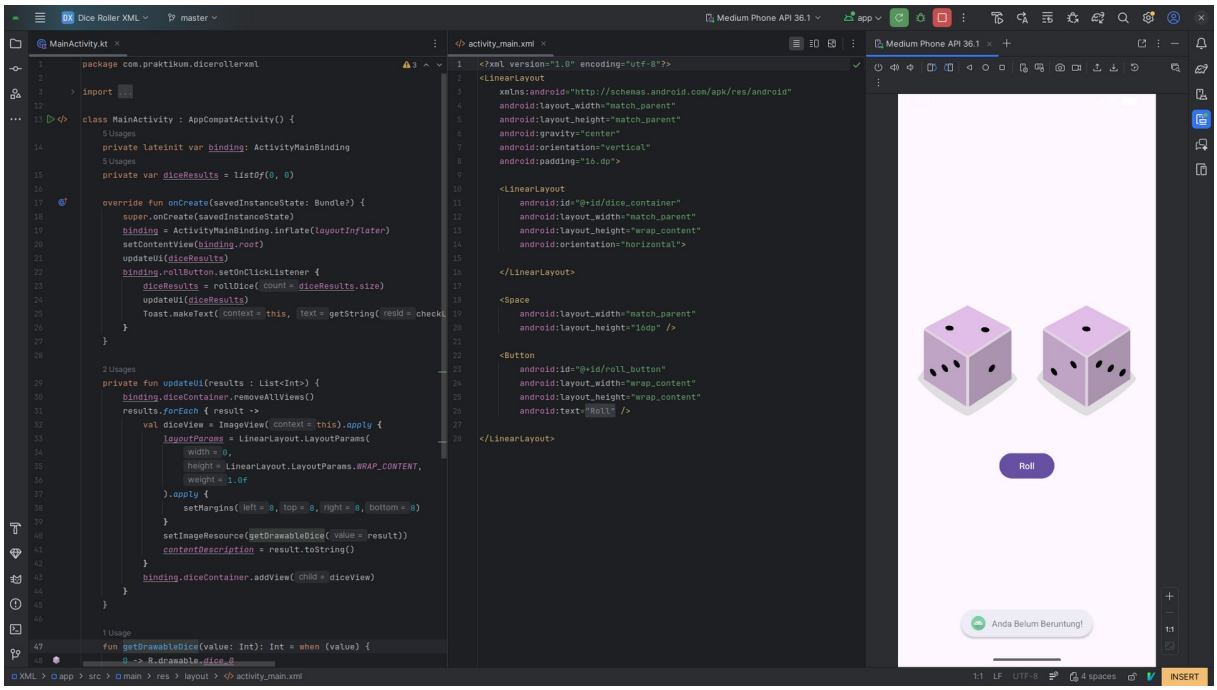
Gambar 7. Screenshot Output Soal 1 Compose Modul 1 (4)



Gambar 8. Screenshot Output Soal 1 Compose Modul 1 (5)



Gambar 9. Screenshot Output Soal 1 XML Modul 1 (1)



Gambar 10. Screenshot Output Soal 1 XML Modul 1 (2)

C. Pembahasan

Saya mengikuti tutorial dari website Tutorial Dice Roll Android Studio yang disediakan oleh asprak, dari tutorial tersebut sebenarnya sampai pada bagian membuat Dice bisa Roll saja tidak sampai ke bagaimana memasukkan lebih dari 1 dice dan melakukan Toast untuk output text melalui alert message.

Kedua implementasi UI menggunakan Compose dan XML memiliki logic code yang sama dan ditulis dengan pendekatan Functional Paradigm

Compose

Terdapat 1 fungsi menarik yaitu onCreate(), fungsi onCreate adalah salah satu callback method pada lifecycle callback yang pertama kali dijalankan ketika activity atau page pertama kali diload. SavedInstanceState adalah parameter yang menyimpan data dari activity sebelumnya, biasanya activity sebelumnya melakukan override pada method onSaveInstanceState().

Method enableEdgeToEdge() adalah method yang digunakan untuk meng-enable edge-to-edge display atau fullscreen tetapi memiliki system bar dengan background transparan. Method SetContent() seperti namanya digunakan untuk menentukan konten dari activity ini.

Terdapat dua method utama yaitu DiceRollerApp() sebagai Preview, dan DiceWithButtonAndImage() sebagai Action atau logic dari aplikasi Dice Roll.

Terdapat tiga method pembantu yaitu getDrawableDice(), rollDice(), dan checkLuck() masing-masing memiliki fungsi yang sama dengan nama methodnya, fungsi ini sebenarnya diambil dari implementasi XML saya (jadi saya menggunakan prinsip DRY), dan untuk mempermudah readibility dari source code.

Pada implementasi ini saya mengutamakan scalability dan readability, saya mengimplementasikan logika UI sedemikian rupa jika di masa depan saya perlu menambahkan satu dice, saya cuma perlu menambahkan satu angka pada variabel diceResult. Saya juga mengutamakan untuk menggunakan paradigma Functional untuk mengurangi side effect. Method rollDice(), getDrawableDice(), dan checkLuck() hanya melakukan kalkulasi dan mengembalikan data baru, tidak mengubah variabel apapun di dalam logikanya.

XML

Menurut saya readibility pada XML sangatlah jelek dibandingkan dengan Compose, dapat dilihat saja pada method onCreate() memiliki lebih banyak isi daripada onCreate() pada

compose, hal ini dikarenakan XML adalah general purpose language jadi memiliki banyak verbosity dan boilerplate di dalamnya.

Terdapat satu method utama yaitu updateUi(), disini adalah logika UI untuk Layout dice_container, gambaran umumnya adalah reset isi Layout dice_container → inisialisasi tiap dice dari list diceResults → untuk tiap dice render sebagai children dari dice_container.

Terdapat tiga method pembantu yaitu getDrawableDice(), rollDice(), dan checkLuck(), sifatnya sama saja dengan yang ada di compose.

Perbedaan Compose dan XML

Terdapat perbedaan besar untuk referensi UI antara Compose dan XML, di XML saya menggunakan view binding untuk mempermudah saya menulis logika untuk Layout XML dan menghilangkan boilerplate harus mencari ID Layout, sedangkan di Compose lebih mudah lagi, tidak ada yang namanya view binding atau ID saya hanya perlu menulis logicnya langsung pada source code.

XML menggunakan Listener jadi dia harus melakukan setOnClickListener() pada Layout roll_button untuk bisa menerima input ketika roll_button ditekan, sedangkan di Compose saya menggunakan onClick pada layout button.

Terakhir, pada Compose saya tidak perlu bolak-balik mengecek layout XML, saya hanya perlu melihat source code activity saat ini yaitu MainActivity, jadi, menurut saya Compose mengurangi waktu saya membaca source code project.

D. Essay

Riset dan *cite* **artikel/jurnal/buku** mengenai Imperative Programming dan Declarative Programming beserta penerapannya di pembuatan UI. Buatkan opini paradigma apa yang kalian prefer berdasarkan riset yang sudah dilakukan. **

Saya prefer menggunakan Paradigma Declarative, walaupun Paradigma Declarative mempunyai kekurangan yaitu black box, kita tidak bisa mengetahui apa yang ada logika pasti dari source code yang menggunakan Paradigma Declarative, tetapi dari perspektif Software Engineer kelebihan yang diberikan oleh Declarative dapat membuat kekurangannya bearable.

Dari studi yang dilakukan oleh Nurhadi dan Aries Muslim disebutkan bahwa Imperative itu bagus untuk visual stability dan memory management, sedangkan Declarative digunakan untuk optimisasi efesiensi CPU [1]. Dari sisi performa Imperative memang lebih baik

daripada Declarative, tetapi kekuatan Declarative tidak berada di performanya tetapi pada Readiblity dan Maintainability.

Salah satu implementasi dari Declarative Paradigm pada UI adalah Jetpack Compose, yang mana berdasarkan study dari Raktinder berhasil mengurangi boilerplate, build dan preview yang lebih cepat dibanding XML dan state management built in yang terintegrasi dengan Kotlin mempermudah context switching, sehingga ia menyatakan bahwa Jetpack Compose atau Declarative Paradigm adalah masa depan dari UI Development [2].

Dari artikel A Practical Introduction to Jetpack Compose Android Apps disebutkan motivasi dibalik Jetpack Compose, salah satunya adalah karena God Object yang bernama View.java yang mana adalah bagian paling penting pada sistem view android sudah mencapai 30 ribu [3].

Apa yang membedakan OOP pada kotlin dengan OOP pada java? *
dari buku Kotlin in Action [4] saya menemukan 6 perbedaan yaitu:

1. Class dan Method pada Kotlin tidak bisa di-inheritance atau dioverride kecuali ditandai open, karena class dan method secara default final, sebaliknya Class dan Method pada Java secara default open, berarti bisa di-inheritance dan override
2. Kotlin dapat memodifikasi variable dari dalam object expression, sedangkan pada Java hanya variabel dengan access modifier final saja yang bisa dimodifikasi
3. Nested Class pada Kotlin tidak punya akses ke field Class di luar kecuali ditandai dengan inner, sedangkan Nested Class pada Java dapat mengakses field Class di luar
4. Kotlin menggunakan val (getter) dan var (setter, getter) untuk menggantikan boilerplate setter dan getter dari Java
5. Interface pada Kotlin bisa memiliki Atribute sedangkan Java hanya bisa memiliki Method saja
6. Class pada Kotlin secara default memiliki access modifier public dan ditandai final, sedangkan pada Java private dan open.

Sebutkan, dan jelaskan mengenai SOLID Principles *
Single Responsibility Principles
Open/Closed Principles
Liskov Principles
Interface Segregation
Depedency Injection

Apa perbedaan antara Class dan Data Class pada Kotlin ?

DAFTAR PUSTAKA

[1] N. Nurhadi and A. Muslim, “Comparative Analysis of the Impact of Declarative and Imperative Programming Approaches on Android Application Performance Through Benchmarking Method,” *J. Res. Soc. Sci. Econ. Manag.*, vol. 5, no. 5, pp. 5234–5248, Dec. 2025, doi: 10.59141/jrssem.v5i5.1231.

[2] R. Singh, R. K. Singh, and A. Singh, “Comparative study of XML vs Jetpack Compose for UI Development in Android,” *Int. J. Innov. Res. Technol. IJIRT*, vol. 12, no. 1, pp. 6175–6184, Jun. 2025.

[3] V. Gaba, “A Practical Introduction to Jetpack Compose Android Apps,” CODE Magazine. Accessed: Apr. 05, 2026. [Online]. Available: https://www.codemag.com/Article/2105061/A-Practical-Introduction-to-Jetpack-Compose-Android-Apps

[4] S. Aigner, R. Elizarov, S. Isakova, and D. Jemerov, *Kotlin in action*, Second edition. Shelter Island: Manning, 2024.